Tryhackme room (Networking Secure protocols)

🔓 Old Internet = basically public diary

- Earlier, data like **passwords, emails, chats** were sent in **plain text**.
- Anyone with a packet sniffer in **promiscuous mode** could just… read everything.
- Users had **zero protection**. Beautiful chaos. Terrible security.

🔐 Birth of SSL → TLS (aka "we should probably fix this")

- Netscape Communications created **SSL** in the 1990s.
- Later, Internet Engineering Task Force developed **TLS** as an improved version.
- Current version: **TLS 1.3 (2018)** → faster + more secure.
- Took **20+ years of improvements** to get here.

🛡️ What TLS actually does

- Works at **Transport Layer (OSI)**.
- Provides:
  - **Confidentiality** → no one can read your data
  - **Integrity** → no one can alter your data
- Without TLS, things like **online banking, shopping, messaging** would be a security nightmare.

🔄 Secure versions of common protocols

TLS upgrades regular protocols into secure ones:

Normal	Secure
HTTP	HTTPS
DNS	DoT (DNS over TLS)
MQTT	MQTTS
SIP	SIPS

That extra **“S” = security**, not decoration.

📜 Certificates (the “trust me bro” system, but verified)

- Servers need a **TLS certificate** to prove identity.
- Steps:
    1. Create a **CSR (Certificate Signing Request)**
    2. Send to a **Certificate Authority (CA)**
    3. CA verifies and signs it
- Your browser trusts servers because it already trusts these CAs.

💸 Paid… unless you’re smart

- Certificates usually cost money 💀
- But Let's Encrypt gives them **for free** (saving the internet from more bad decisions)

⚠️ Self-signed certificates

- Anyone can create one
- But:
    - No trusted third party verifies it
    - So it’s basically: *“trust me, I’m legit”*
    - Browsers don’t trust it (and honestly, fair)

HTTP (Normal, Insecure)

After DNS resolves the domain:

1. **TCP Handshake**
    - Client and server establish connection (3-way handshake).
2. **HTTP Communication**
    - Client sends requests (e.g., `GET / HTTP/1.1`)
    - Server responds with data.
3. **Connection Termination**
    - TCP connection is closed.

👉 Key point:
All data is **readable in plain text** (easy to intercept).

HTTPS (HTTP over TLS)

After DNS resolution:

1. **TCP Handshake**
    - Same as HTTP.
2. **TLS Handshake**
    - Client and server establish encryption keys.
    - Secure session is created.
3. **HTTP Communication (Encrypted)**
    - Same HTTP requests/responses, but now encrypted.
    - Appears as “Application Data” in tools like Wireshark.

What Changes with TLS?

- Without TLS → You can read packets directly.
- With TLS → Data appears as **gibberish** unless you have the decryption key.
- If the key is provided (e.g., in testing), tools like Wireshark can decrypt and show actual HTTP data.

Key Takeaway

- TLS adds **security (encryption)** on top of HTTP.
- It **does not change TCP/IP or HTTP itself**.
- It simply wraps HTTP traffic so it cannot be read or modified by attackers.

Short version:
HTTP = readable traffic
HTTPS = same traffic, but locked inside encryption

That’s it. Same conversation, just now nobody else gets to eavesdrop on it.

Adding TLS to SMTP, POP3, and IMAP is no different than adding TLS to HTTP. Similar to how HTTP gets an appended S for *Secure* and becomes HTTPS, SMTP, POP3, and IMAP become SMTPS, POP3S, and IMAPS, respectively. Using these protocols over TLS is no different than using HTTP over TLS; therefore, almost all the points from the HTTPS discussion apply to these protocols.

The insecure versions use the default TCP port numbers shown in the table below:

Protocol	Default Port Number
HTTP	80
SMTP	25
POP3	110
IMAP	143

The secure versions, i.e., over TLS, use the following TCP port numbers by default:

Protocol	Default Port Number
HTTPS	443
SMTPS	465 and 587
POP3S	995
IMAPS	993

TLS can be added to many other protocols; the reasoning and advantages would be similar.

Telnet was widely used for remote login, but it sends all data, including usernames and passwords, in cleartext. This makes it very easy for attackers to capture credentials by monitoring network traffic.

To solve this problem, Tatu Ylönen developed the Secure Shell protocol in 1995. A more secure version, SSH 2, followed in 1996. Later, OpenBSD developers released OpenSSH in 1999, which is now the most commonly used implementation of SSH.

Key Features of SSH and OpenSSH

Secure authentication
SSH supports multiple authentication methods such as passwords, public key authentication, and two factor authentication.

Confidentiality
All communication is encrypted, preventing attackers from reading the data. It also warns users about unknown server keys to reduce man in the middle attacks.

Integrity
Encryption ensures that data cannot be altered during transmission.

Tunneling
SSH can securely carry other protocols, creating a tunnel similar to a VPN.

X11 forwarding
It allows running graphical applications on a remote system and displaying them locally.

Usage

To connect to a remote system
ssh username@hostname
If the username is the same as the local system, only ssh hostname is needed.
Authentication can be done using a password or automatically using public keys.

Ports

Telnet uses port 23
SSH uses port 22

Key Takeaway

SSH replaced Telnet by providing secure, encrypted communication for remote access, making it safe from eavesdropping and data tampering.

SFTP (SSH File Transfer Protocol)

- Part of the SSH suite and uses **port 22**.
- Provides **secure file transfer over SSH**.
- Easy to use if OpenSSH is enabled.
- Commands are Unix-like, such as `get` (download) and `put` (upload).

FTPS (FTP Secure)

- Traditional FTP secured using **TLS**.

- Uses **port 990** (instead of FTP's port 21).
- Requires a **TLS certificate** for security.
- More complex to configure, especially with firewalls, because it uses separate connections for control and data.

Key Difference

- **SFTP** uses **SSH** for security and is simpler to set up.
- **FTPS** uses **TLS** like HTTPS and needs certificates, making it more complicated.

Takeaway

SFTP is generally easier and more straightforward, while FTPS is more complex due to TLS setup and connection handling, even though both provide secure file transfer.

A **VPN (Virtual Private Network)** lets a company connect multiple offices or remote users over the Internet so they can access resources as if they were on the same local network.

Why VPN is needed

- The Internet (TCP/IP) only ensures **data delivery**, not **security**.
- It does **not protect data from being read or modified**.
- VPN solves this by adding **privacy and security**.

How VPN works

- A **VPN client** connects to a **VPN server**.
- A secure **encrypted tunnel** is created.
- Data sent through this tunnel is **encrypted before sending** and **decrypted at the destination**.

Two common setups:

- **Site-to-site VPN** → connects entire branch offices
- **Remote access VPN** → connects individual users (like employees working from home)

What happens after connecting

- Most traffic is routed through the VPN tunnel.
- Websites see the **VPN server's IP address**, not your real one.
- Your ISP can only see **encrypted traffic**, not what you're doing.

Extra points

- VPN can be used to **bypass geographical restrictions**.
- Some VPNs may **leak your real IP or DNS**, so testing is important.
- Not all VPNs route all traffic; some only give access to internal networks.
- VPN usage may be **restricted or illegal in some countries**.

Key takeaway

A VPN creates a secure, private connection over the public Internet, making remote access and inter-office communication safe and flexible.